Lab 1: Indexing

1. Introduction to Indexing

An **index** is a data structure that enhances the speed of data retrieval in a database table.It allows SQL Server to access specific rows efficiently without scanning the entire table.

Analogy: A database index functions like the index of a book rather than reading every page, you locate the topic and go directly to the correct page number.

Benefits: - Accelerate queries using SELECT, JOIN, and ORDER BY.
- Improve response time for large or frequently accessed tables.

Limitations: - Require additional storage space.
- Can slow down INSERT, UPDATE, and DELETE operations due to index maintenance.

How Indexes Work Internally

SQL Server implements indexes using a **B-tree (balanced tree)** structure that organizes data hierarchically for efficient searching.

- The **root node** begins the search process.
- **Intermediate nodes** direct the lookup path.
- **Leaf nodes** store the final data references:
    - In a **clustered index**, leaf nodes hold the actual data rows.
    - In a **non-clustered index**, leaf nodes contain pointers to those rows.

This design enables SQL Server to locate data quickly by traversing the tree instead of scanning the entire table sequentially.

2. Types of Indexes

Type	Description	Key Points
Clustered Index	Physically sorts and stores the table's data rows based on the indexed column.	One per table only. Usually created automatically on the PRIMARY KEY.
Non-Clustered Index	Stores key values separately from the actual data and uses pointers to locate rows.	You can create multiple per table. Best for frequently searched columns.
Composite Index	Built on two or more columns.	Useful when queries filter or sort by multiple columns.

3. Create the Table

We'll use the following table to explore indexing concepts:

```
CREATE TABLE Employees (
    EmployeeID INT IDENTITY(1,1) ,
    Name NVARCHAR(50),
    Department NVARCHAR(50),
    Salary INT
);

Insert Sample Data:

INSERT INTO Employees (Name, Department, Salary)
VALUES
('Ali', 'Finance', 8500),
('Sara', 'IT', 12000),
('Omar', 'Finance', 9400),
('Mona', 'HR', 7700),
('Laila', 'IT', 11000),
('Youssef', 'Marketing', 9500);
```

4. Working with Indexes

4.1 Clustered Index

A clustered index defines the physical order of data in a table. Most tables already have one automatically created on the PRIMARY KEY.

```
CREATE CLUSTERED INDEX IX_Employees_EmployeeID
ON Employees(EmployeeID);
```

Note: A table can only have one clustered index because it defines the physical order of rows.

4.2 Non-Clustered Index

A non-clustered index creates a separate structure that points to the physical data location. It is typically used to improve lookups on frequently searched columns, such as Department.

```
CREATE NONCLUSTERED INDEX IX_Employees_Department
ON Employees(Department);
```

This index enables SQL Server to quickly locate employees in a specific department using an Index Seek, instead of scanning the entire table (Table Scan).

4.3 Composite Index

A composite index includes two or more columns. It is ideal when queries frequently use multiple conditions, such as Department and Salary.

```
-- Create a composite index on two columns
CREATE NONCLUSTERED INDEX IX_Employees_Department_Salary
ON Employees (Department, Salary);
```

Note: The order of columns in a composite index matters (Department, Salary) is not the same as (Salary, Department).

5. Viewing and Managing Indexes

View all indexes on a table:

```
EXEC sp_helpindex 'Employees';
```

- This will list: Index name, Index description (clustered or nonclustered), Indexed columns.
- Alternatively, view indexes from SSMS under the table → Indexes folder.

Remove an unnecessary index:

```
DROP INDEX IX_Employees_Department ON Employees;
```

Tasks:

1. Create a non-clustered index on Salary.
2. View all indexes using sp_helpindex.
3. Drop an index you created.